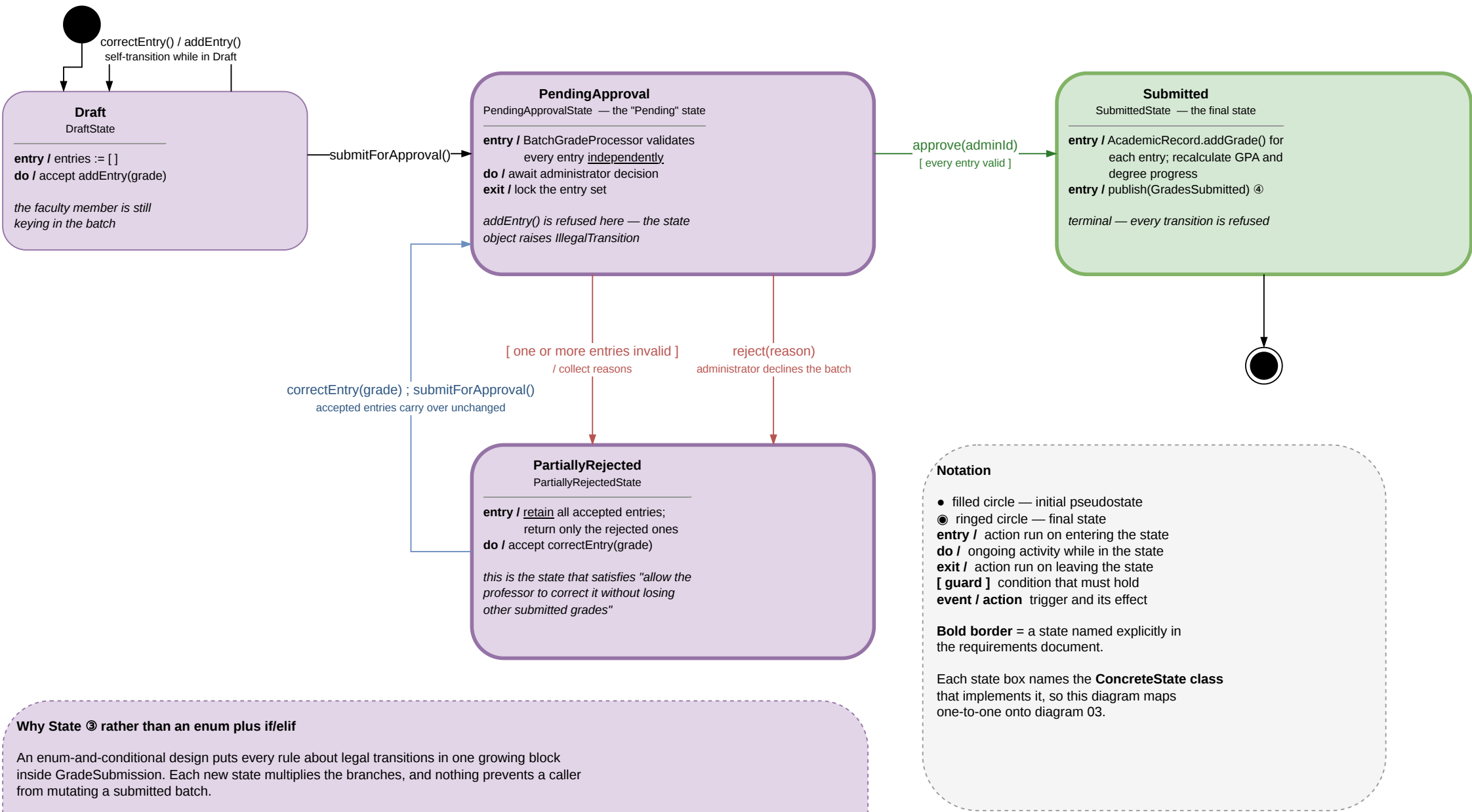


State Diagram — GradeSubmission lifecycle

Requirement: "The system must have a process for grade approval (e.g. a "Pending" state before a final "Submitted" state)." Realised with the State pattern ③.



Why State ③ rather than an enum plus if/elif

An enum-and-conditional design puts every rule about legal transitions in one growing block inside GradeSubmission. Each new state multiplies the branches, and nothing prevents a caller from mutating a submitted batch.

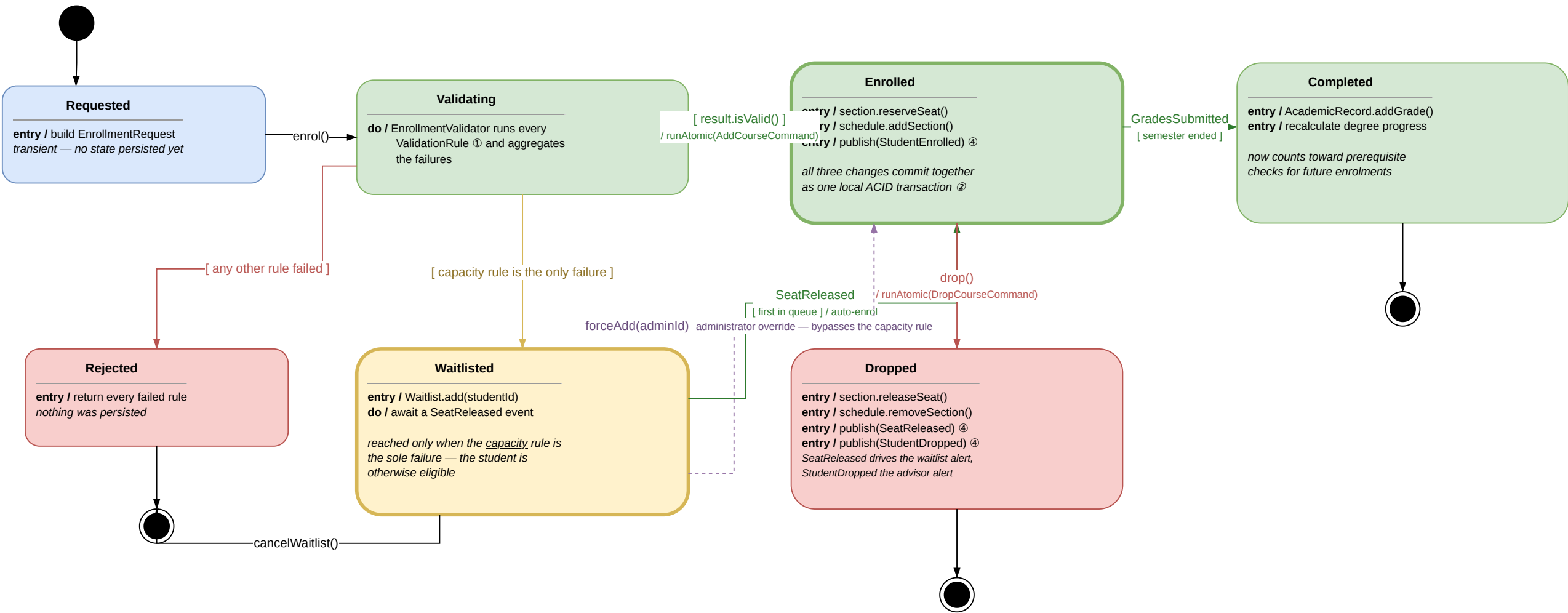
With the State pattern each ConcreteState answers only for itself:

- **DraftState** permits `addEntry()` and `submitForApproval()`
- **PendingApprovalState** permits `approve()` and `reject()`, and refuses `addEntry()`
- **PartiallyRejectedState** permits `correctEntry()` while keeping accepted grades
- **SubmittedState** refuses everything

The base class default raises `IllegalTransition`, so an unhandled operation fails loudly instead of corrupting the batch. Adding an "UnderAudit" state later is a new class — the Context is untouched (Open/Closed Principle).

State Diagram — Enrollment lifecycle

The lifecycle of the central business object. Transitions correspond to the EnrollmentStatus enumeration on diagram 02 and to the Command operations on diagram 03.



Reading this diagram against the requirements

Validating → **Enrolled** is the happy path of the §3.1 use case: "check if the student has the necessary prerequisites... check if there's available capacity... the enrolment is confirmed, and the student's schedule and academic record are updated." All three entry actions on **Enrolled** commit as one transaction.

Validating → **Waitlisted** vs. **Validating** → **Rejected** is a deliberate design distinction. A student who fails only the capacity check is eligible and is queued; a student missing a prerequisite is not eligible and must be refused. Collapsing these into one "failed" state would make the waitlist requirement unimplementable.

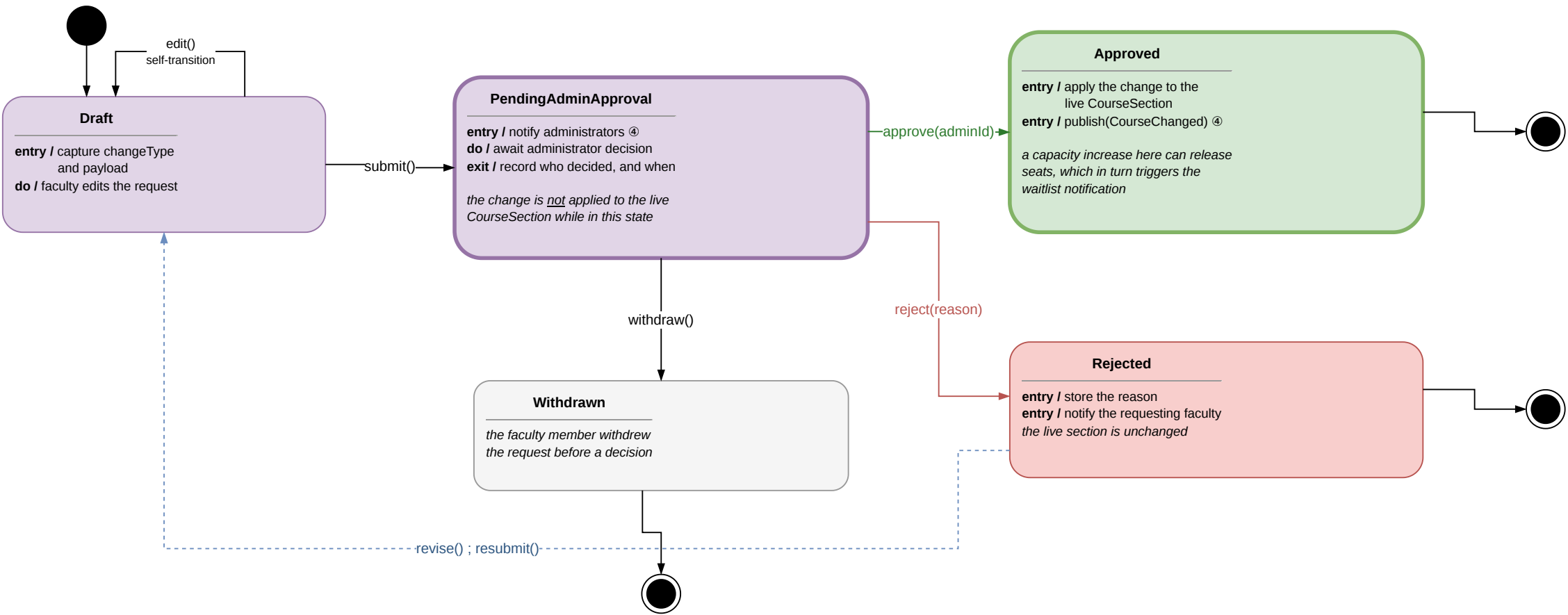
Dropped publishes StudentDropped, which is what drives the automated waitlist alert in diagram 05 page 2.

Completed feeds back into **PrerequisiteRule**: a completed course becomes an input to future prerequisite checks, closing the loop between the Academic Records and Enrolment contexts.

The dashed purple transition is the administrator override from §3.3 — "force-add a student into a full class" — modelled as a legal transition rather than a special case buried in code.

State Diagram — CourseChangeRequest lifecycle

Requirement: "Instructors can submit requests to update course descriptions, add prerequisites, or change course capacity (these requests must be approved by an administrator)."



Design points worth stating in the report

- 1. The request is a first-class object, not a flag.** Modelling the approval workflow as its own entity with its own lifecycle keeps the audit trail (who requested what, who decided, when, and why) without polluting CourseSection with approval fields. Single Responsibility Principle at the entity level.
- 2. The live section is only mutated on entry to Approved.** While the request sits in PendingAdminApproval the published catalogue is unaffected, so students never see a capacity or prerequisite that has not been authorised. This is the whole point of the requirement that these requests "must be approved by an administrator."
- 3. The same State pattern ③ implementation serves two entities.** GradeSubmission and CourseChangeRequest both follow a Draft → Pending → terminal shape, so the pattern is applied twice with different ConcreteState families. Reusing one design idea across two workflows is the DRY principle operating at the design level rather than the code level.
- 4. Approving a capacity increase cascades.** The entry action publishes CourseChanged; a capacity increase releases seats, and the existing WaitlistObserver ④ picks that up with no extra code. Two independently designed features compose because both are event-driven.